Breeze Payment Unity SDK

Unity 6.3+

version 1.1.0

The Breeze Payment Unity SDK enables seamless payment integration for Unity games on iOS and Android platforms. Show native payment option dialogs, handle payment flows, and provide a smooth checkout experience for your players.

Features

- 🎮 **Native Integration** — Seamlessly integrated with Unity for iOS and Android
- 💳 **Payment Options Dialog** — Native bottom-sheet dialog with product info, pricing, and save badges
- 🌐 **Payment Webview** — Open the payment page directly in an in-app webview, skipping method selection
- 🎨 **Theming** — Light, dark, and auto (follows system) themes on both platforms
- 🛠️ **Easy Setup** — Install via Unity Package Manager with one URL

Requirements

- Unity 6.3 LTS or later
- iOS 15.0+ (for iOS builds)
- Android API 21+ (for Android builds)
- .NET Standard 2.1 or .NET Framework 4.8

Installation

Unity Package Manager (Recommended)

1. Open your Unity project
2. Open **Window** → **Package Manager**
3. Click the **+** button in the top-left corner
4. Select **Add package from git URL**

5. Paste the following URL:

   `https://github.com/breeze-com/breeze-unity.git?path=sdks/Unity/Breeze`

6. Click **Add**

The SDK will be automatically installed along with its dependencies (Newtonsoft.Json).

Quick Start

1. Initialize the SDK

Initialize Breeze in your game's startup code (e.g., in a MonoBehaviour's `Start()` method):

```
using UnityEngine;

public class GameManager : MonoBehaviour
{
    void Start()
    {
        Breeze.Initialize(new BreezeConfiguration()
        {
            AppScheme = "mygame://",  // Your app's custom URL scheme
            Environment = BreezeEnvironment.Production,
        });
    }

    void OnDestroy()
    {
        Breeze.Uninitialize();
    }
}
```

> **Note:** `Initialize()` can only be called once. Call `Uninitialize()` first if you need to re-initialize with different settings.

2. Show Payment Options Dialog

Display a payment options dialog when the user wants to make a purchase:

```
void ShowPaymentDialog()
{
    var request = new BrzShowPaymentOptionsDialogRequest()
    {
        Title = "Select payment method",
        ProductDisplayInfo = new BrzProductDisplayInfo()
        {
            DisplayName = "Premium Pack",
            OriginalPrice = "USD $9.99",
            BreezePrice = "USD $7.99",
            Decoration = "Save 20%",
            ProductIconUrl = "https://example.com/icon.png",
        },
        DirectPaymentUrl = "https://pay.breeze.cash/page_xxx/pcs_xxx",
        Data = "product-id-123",  // Optional: pass-through data returned on dismiss
        Theme = BrzPaymentOptionsTheme.Auto,  // Auto, Light, or Dark
    };

    // Subscribe to dismissal events
    Breeze.Instance.OnPaymentOptionsDialogDismissed += OnPaymentDialogDismissed;

    // Show the dialog
    Breeze.Instance.ShowPaymentOptionsDialog(request);
}

void OnPaymentDialogDismissed(BrzPaymentDialogDismissReason reason, string data)
{
    // Unsubscribe first to avoid duplicate handlers
    Breeze.Instance.OnPaymentOptionsDialogDismissed -= OnPaymentDialogDismissed;

    switch (reason)
    {
        case BrzPaymentDialogDismissReason.CloseTapped:
            // User closed the dialog without selecting a payment method
            break;
        case BrzPaymentDialogDismissReason.DirectPaymentTapped:
            // User selected Breeze direct payment — browser opened
            // Start polling for payment confirmation (see step 4)
            break;
        case BrzPaymentDialogDismissReason.AppStoreTapped:
            // User selected App Store / Google Play — trigger IAP purchase
            break;
```

```
    }
}
```

3. Show Payment Webview (Alternative)

Instead of a payment options dialog, you can open the Breeze payment page directly inside an in-app webview. This skips the payment method selection step and takes the player straight to the payment page.

```
async Awaitable ShowPaymentWebview()
{
    // Create an order on your game server first
    var result = await gameClient.CreateOrderAsync(new CreateOrderInput
    {
        ProductId = "prd_your_product_id",
        Quantity = 1,
    });

    var request = new BrzShowPaymentWebviewRequest()
    {
        DirectPaymentUrl = result.Data.Url,  // URL from your server
        Data = "product-id-123",             // Optional: pass-through data returned on
    };

    // Subscribe to dismissal event
    Breeze.Instance.OnPaymentWebviewDismissed += OnPaymentWebviewDismissed;

    // Show the webview
    Breeze.Instance.ShowPaymentWebview(request);
}

void OnPaymentWebviewDismissed(BrzPaymentWebviewDismissReason reason, string data)
{
    // Unsubscribe first to avoid duplicate handlers
    Breeze.Instance.OnPaymentWebviewDismissed -= OnPaymentWebviewDismissed;

    switch (reason)
    {
        case BrzPaymentWebviewDismissReason.PaymentSuccess:
            // Payment completed — verify server-side before granting items
            StartPaymentVerification();
            break;
        case BrzPaymentWebviewDismissReason.PaymentFailure:
            // Payment failed or was cancelled
            break;
        case BrzPaymentWebviewDismissReason.Dismissed:
            // User closed the webview manually
            break;
        case BrzPaymentWebviewDismissReason.LoadError:
            // Webview failed to load the payment page
            break;
```

```
    }
}
```

> **Note:** Unlike `ShowPaymentOptionsDialog`, the webview handles the entire payment flow natively — no deep link handling or `DismissPaymentPageView()` call is needed.

4. Set Up Deep Links

Breeze uses deep links to return the player to your app after payment:

- `<your-app-scheme>://breeze-payment/purchase/success`
- `<your-app-scheme>://breeze-payment/purchase/failure`

Setup:

1. Follow the Unity deep linking guide
2. After generating `Unity-iPhone.xcodeproj`, verify in Xcode:
   - `Info.plist` → `URL Types` → `Item 0` → `URL Schemes` → `Item 0` = your scheme (e.g. `mygame`, **without** `://`)

Create payment page URLs on your server with the correct redirect URLs:

```
{
  "successReturnUrl": "mygame://breeze-payment/purchase/success",
  "failReturnUrl": "mygame://breeze-payment/purchase/failure"
}
```

See: Create a Payment Page

Handle the deep link in Unity:

```
void Start()
{
    Application.deepLinkActivated += OnPaymentPageResult;
}

void OnPaymentPageResult(string url)
{
    // Dismiss the in-app browser (iOS only, no-op on Android)
    Breeze.Instance.DismissPaymentPageView();

    // IMPORTANT: Do NOT grant items based on the URL alone!
    // Deep links can be spoofed. Always verify server-side first.
    StartPaymentVerification();
}
```

Testing deep links on iOS Simulator:

```
xcrun simctl openurl booted "mygame://breeze-payment/purchase/success"
```

5. Verify Payment (Recommended)

⚠️ **Never grant items based on the deep link URL alone.** Deep links can be spoofed by any app on the device. Always verify the payment status with your game server.

Payment flow diagram:

```
Player taps "Pay" → Breeze browser opens → Player pays
                                                ↓
                                Breeze webhook → Your server marks order paid
                                                ↓
          Verify the payment with your server → Server returns "succeeded"
                                                ↓
                                Game grants items ✅
```

6. Recovery on App Restart

If the game is killed during payment, the Breeze webhook still fires and your server knows the order is paid. But the client never polled, so items aren't granted in that session.

Recommendation: On app startup, check your server for any pending fulfilled orders:

```
async void Start()
{
    // Check for orders that were paid while the app was closed
    var pendingOrders = await gameClient.GetPendingFulfilledOrders();
    foreach (var order in pendingOrders)
    {
        GrantItems(order);
        await gameClient.AcknowledgeOrder(order.Id);
    }
}
```

Theming

The payment dialog supports three themes:

Theme	Behavior
`BrzPaymentOptionsTheme.Auto`	Follows system dark/light mode
`BrzPaymentOptionsTheme.Light`	Always light background
`BrzPaymentOptionsTheme.Dark`	Always dark background

Both iOS and Android render the dialog programmatically with matching designs — no storyboards or XML layouts.

Platform Notes

iOS

- Payment browser uses `SFSafariViewController` (in-app, shares cookies with Safari)
- `DismissPaymentPageView()` programmatically closes the Safari view when the deep link returns
- `StoreKit.Storefront` is checked — if the user's App Store region is not USA, the dialog auto-dismisses with `AppStoreTapped` (configurable behavior for compliance)

Android

- Payment browser uses Chrome Custom Tabs (detected via reflection, no `androidx.browser` dependency required)

- Falls back to the default browser if Custom Tabs are unavailable
- `DismissPaymentPageView()` is a no-op on Android (Custom Tabs run in a separate process)
- The dismiss reason `GoogleStoreTapped` is used instead of `AppStoreTapped`

Editor

- All methods succeed but don't show native UI
- `GetDeviceUniqueId()` returns a persistent GUID stored in `PlayerPrefs`
- Useful for testing flow logic without building to device

Debug Logging

SDK logs are **stripped by default** in production builds. To enable verbose logging during development:

1. Open **Edit** → **Project Settings** → **Player** → **Other Settings**
2. Under **Scripting Define Symbols**, add: `BREEZE_DEBUG`
3. Click **Apply**

This enables `Debug.Log` calls that show request JSON, dismiss reasons, and verification progress. **Remove `BREEZE_DEBUG` before shipping** — logged data may include payment URLs and custom data.

Security

The SDK implements several security measures:

Measure	Details
Host validation	`directPaymentUrl` must have a host ending in `.breeze.cash` (iOS + Android native)
No client-side payment trust	Deep link URLs are **not** authoritative — always verify via webhook + server
Log stripping	Request JSON not logged unless `BREEZE_DEBUG` is defined
URL encoding	Order IDs are escaped with `Uri.EscapeDataString` to prevent path traversal

⚠️ Deep Link Security

Custom URL schemes (`mygame://`) are inherently insecure — any app can register the same scheme and intercept the redirect. **Never trust the deep link URL as proof of payment.** The webhook → server → client polling flow is the only secure path.

API Reference

`Breeze`

Method	Description
`Breeze.Initialize(config)`	Initialize the SDK. Must be called once before any other method.
`Breeze.Uninitialize()`	Clean up SDK resources. Call before re-initializing.
`Breeze.Instance`	Singleton instance (null if not initialized).
`Instance.ShowPaymentOptionsDialog(request)`	Show the native payment options dialog.
`Instance.ShowPaymentWebview(request)`	Open the payment page directly in an in-app webview.
`Instance.DismissPaymentPageView()`	Dismiss the in-app payment browser (iOS). No-op on Android/Editor.
`Instance.GetDeviceUniqueId()`	Returns a platform-specific device ID.
`Instance.OnPaymentOptionsDialogDismissed`	Event fired when the payment dialog is dismissed.
`Instance.OnPaymentWebviewDismissed`	Event fired when the payment webview is dismissed.

Enums

Enum	Values
`BrzPaymentDialogDismissReason`	`CloseTapped` , `DirectPaymentTapped` , `AppStoreTapped` , `GoogleStoreTapped`

Enum	Values
`BrzPaymentWebviewDismissReason`	`Dismissed` , `PaymentSuccess` , `PaymentFailure` , `LoadError`
`BrzPaymentOptionsTheme`	`Auto` , `Light` , `Dark`
`BrzPaymentStatus`	`Pending` , `Succeeded` , `Failed` , `Expired` , `Refunded` , `Unknown`
`BrzShowPaymentOptionsResultCode`	`Success` , `NullInput` , `InvalidUtf8` , `JsonDecodingFailed`
`BrzShowPaymentWebviewResultCode`	`Success` , `NullInput` , `InvalidUtf8` , `JsonDecodingFailed` , `InvalidUrl`

Example Project

A complete example is in `examples/UnityBreezeDemo/` :

- `ShowPaymentOptionsDialogUI.cs` — Full UI flow with dialog, deep link handling, and IAP fallback
- `YourGameClient.cs` — Example game server client for creating orders
- `IapManager.cs` — Unity IAP integration for App Store/Google Play fallback

Tests

The SDK includes unit tests in `sdks/Unity/Breeze/Tests/Runtime/` :

Test File	Coverage
`TestBreezeConfiguration`	Initialization, validation, singleton lifecycle
`TestBreezeHelper` / `Expanded`	URL building, Base64, query strings, region detection
`TestBreezeNativeModels` / `Expanded`	JSON serialization, enum values, edge cases
`TestBreezeSingleton`	Init/uninit, re-init, cleanup
`TestBreezeIntegration`	End-to-end payment flows

Test File	Coverage
`TestBreezeSecurity`	URL validation, HTTPS enforcement
`TestBreezeNativeNoop`	Editor noop: DismissPaymentPageView no-op, device ID stability, interface compliance
`TestAndroidCallbackParsing`	Android bridge int vs string reason parsing, dismiss reason enum coverage

Run tests via **Window** → **General** → **Test Runner** in Unity.

Troubleshooting

SDK Not Initialized

If you see `"BreezePayment already initialized"`, call `Breeze.Uninitialize()` before re-initializing.

Payment Dialog Not Appearing

- Verify `Breeze.Initialize()` was called with a valid `AppScheme`
- Ensure you're running on a physical device or simulator (Editor uses Noop implementation)
- Check that `directPaymentUrl` host ends with `.breeze.cash`

Android: Chrome Custom Tabs Not Opening

- Chrome or a Custom Tabs-compatible browser must be installed
- The SDK falls back to the default browser automatically
- Check `adb logcat` for `BreezeNativeAndroid` tags (enable `BREEZE_DEBUG`)

iOS: Dialog Auto-Dismisses

- This may happen if the user's App Store storefront is not USA
- The SDK checks `StoreKit.Storefront.current` and auto-dismisses with `AppStoreTapped` for non-US users
- This is intentional for compliance — modify `BreezePaymentOptionsDialog.swift` if expanding to other regions

Changelog

v1.1.0

- **Added** `Breeze.Instance.ShowPaymentWebview` show payment page in webview instead of browser tabs
- **Added** `BREEZE_DEBUG` compile flag — SDK logs stripped by default
- **Added** `BrzPaymentOptionsTheme` support (auto/light/dark)
- **Added** `AppScheme` validation on initialization
- **Fixed** Android dismiss reasons all reported as `CloseTapped` (type mismatch in bridge)
- **Fixed** Duplicate Android callbacks (two competing receivers)
- **Fixed** iOS double callback on direct payment tap
- **Fixed** `Breeze.Initialize()` silently re-initialized when called twice
- **Fixed** `DismissPaymentPageView()` crash in Editor (was `NotImplementedException`)
- **Fixed** Android `getDeviceUniqueId()` returned hardcoded string
- **Tests** 8 test files covering all SDK components

v1.0.0

- Initial release

Support

For issues, questions, or feature requests:

- **Email:** support@breeze.cash
- **Website:** https://breeze.cash
- **GitHub Issues:** Create an issue

Made with ❤️ by Breeze